

Redes em Java

Conceitos fundamentais explicados de forma simples e direta através de um sistema multiplayer.

Fundamentos de Identificação

Antes de transmitir dados, precisamos saber **quem** somos na rede e **onde** os outros estão.

Identificação: IP e Interfaces

InetAddress

Representa um endereço IP (IPv4 ou IPv6). No Java, usamos para traduzir nomes de domínio ou obter o IP local da máquina.

```
InetAddress.getHostAddress()
```

NetworkInterface

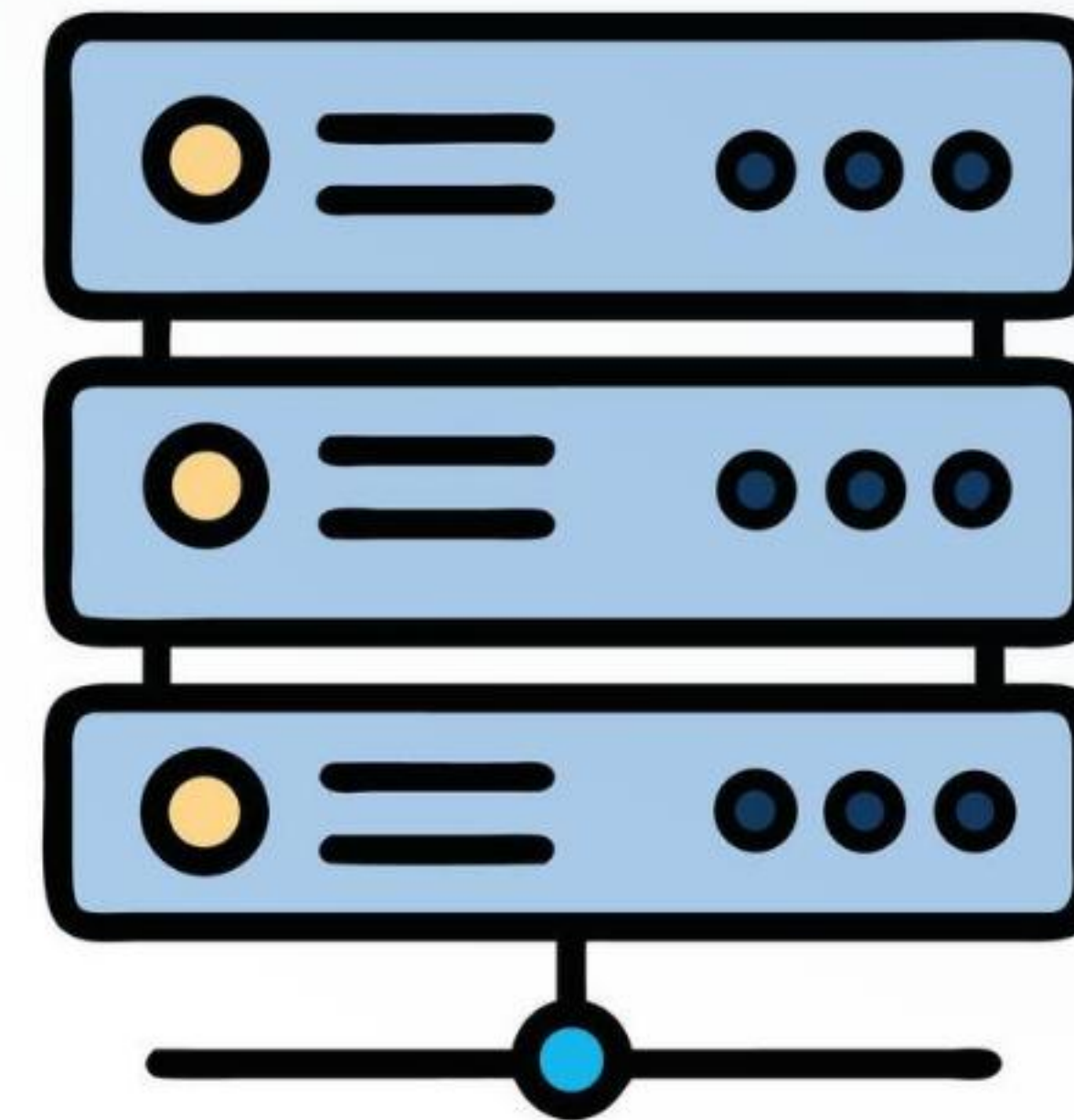
Permite listar todas as placas de rede (Wi-Fi, Ethernet) físicas e virtuais conectadas ao sistema operacional.

```
NetworkInterface.getNetworkInterfaces()
```


O Ouvvinte: ServerSocket

O **ServerSocket** é o ponto central do servidor. Ele "escuta" uma porta específica (ex: 5000) e aguarda conexões.

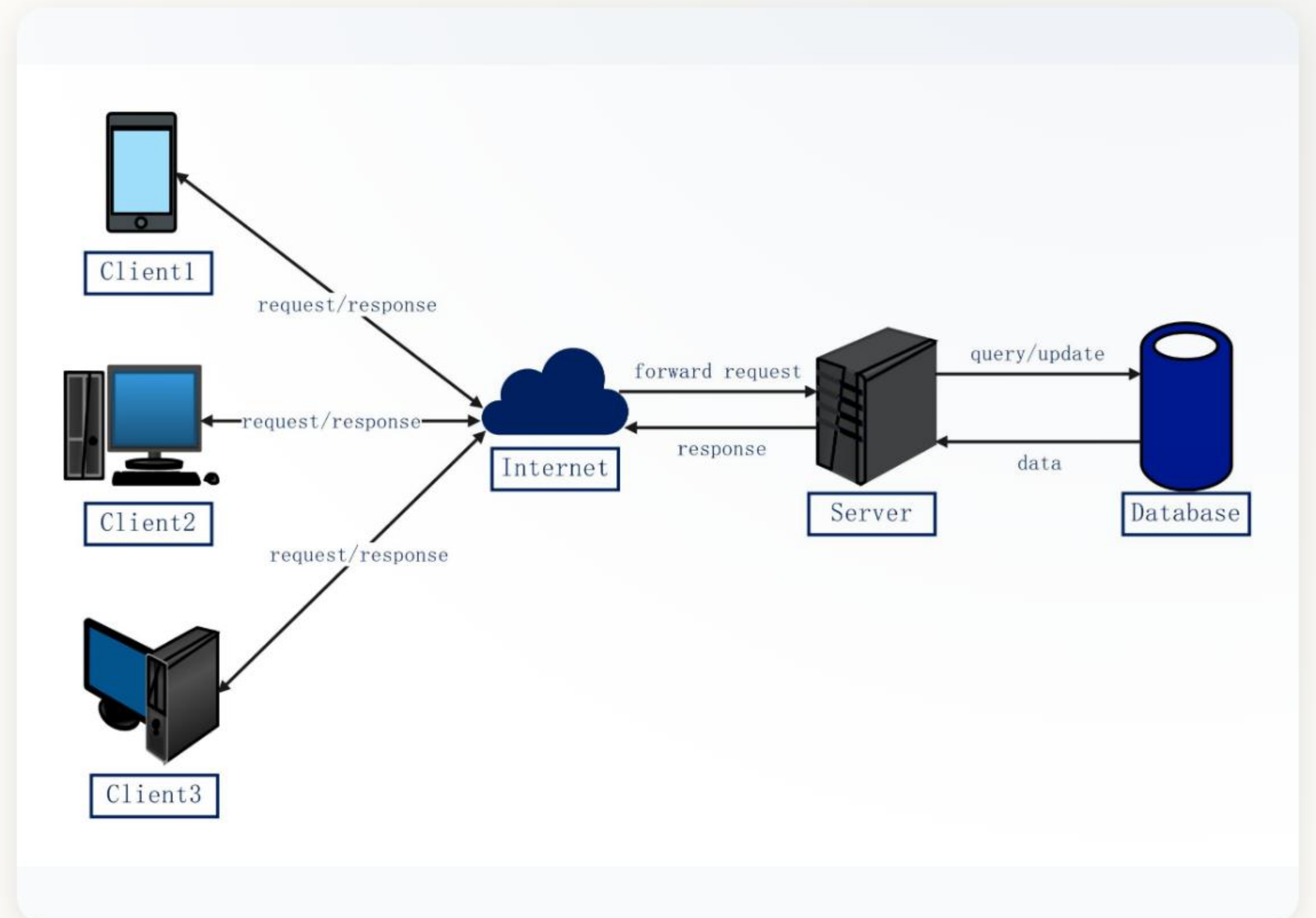
- ✓ **accept():** Bloqueia a execução até que um cliente se conecte.
- ✓ **Bind:** Associa o processo a uma porta TCP única.
- ✓ **Autoridade:** Garante que o servidor gerencie as regras do jogo.



O Conector: Socket

No lado do Cliente, o **Socket** tenta estabelecer uma ponte bidirecional com o IP e Porta do servidor.

- **Host:** O destino (ex: "192.168.0.10").
- **Porta:** O canal de entrada do servidor.
- **Handshake:** Negociação automática que garante a conexão TCP estável.



Fluxo de Dados: Streams I/O



BufferedReader

Lê dados de entrada do socket de forma eficiente. No jogo, ele escuta as mensagens enviadas pela rede.



PrintWriter

Envia strings e dados formatados pelo socket. O parâmetro `autoFlush` garante o envio imediato.



Bidirecional

O Socket fornece um `InputStream` para leitura e um `OutputStream` para escrita simultânea.

Protocolo: A Linguagem Comum

Comando	Sentido	Propósito no Jogo
LOGIN	Cliente -> Servidor	Identifica o jogador no início da sessão.
ACAO	Cliente -> Servidor	Envia a escolha do alvo (si mesmo ou oponente).
VEZ_DE	Servidor -> Cliente	Notifica qual cliente deve agir agora.
RESULTADO	Servidor -> Cliente	Informa se o disparo atingiu o alvo ou falhou.

Usamos delimitadores como ";" para separar o comando dos dados.

Concorrência e Threads

1

Thread por Cliente

O servidor Java é multi-threaded. Para cada nova conexão, criamos uma `Thread` ou usamos um `ExecutorService`.

 **Não-bloqueante:** Enquanto uma thread aguarda dados de um cliente, o servidor aceita outros.

 **ClientHandler:** Classe que implementa `Runnable` para gerenciar a lógica individual.

Ciclo de Vida da Conexão

1. Escuta

Servidor inicia o
`ServerSocket`.

3. Troca

I/O via Streams
(Protocolo).

2. Conexão

Cliente instancia
`Socket` e conecta.

4. Encerramento

Socket é fechado
recursos liberados



Gerenciamento de Partida

- ☰ **Fila de Espera:** O servidor agrupa clientes (ex: 2 em 2) para iniciar o jogo.
- 🕒 **Timeout:** Uso de `ScheduledExecutorService` para ações automáticas (AFK).
- 🛡️ **Autoridade:** Apenas o servidor decide se a bala foi disparada ou não.
- 🔄 **Sincronização:** Métodos `synchronized` evitam conflitos de dados em múltiplas threads.

Broadcasting: Comunicação em Grupo



Jogador A



Mensagem para todos



Jogador B

O servidor mantém uma lista de conexões e percorre-a para enviar atualizações de estado a todos os envolvidos.

Perguntas?

Utilize os pacotes `java.net` e `java.io` para criar suas próprias aplicações distribuídas.

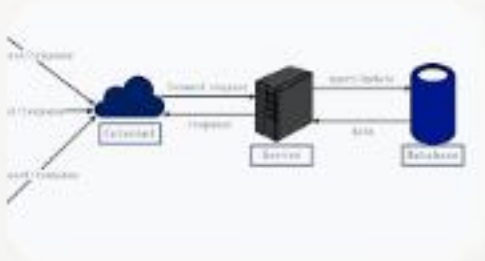
 Desenvolvido com Java Standard Edition

Image Sources



https://static.vecteezy.com/system/resources/previews/065/410/964/non_2x/minimalist-server-rack-illustration-with-light-blue-white-background-vector.jpg

Source: www.vecteezy.com



<https://edrawmax.wondershare.com/templates/Network/network-diagram-for-client-server.jpg>

Source: edrawmax.wondershare.com



<https://cdn-icons-png.flaticon.com/512/9524/9524131.png>

Source: www.flaticon.com